are immediate children of a ... you get the idea. I'm hoping that you might see some parallels between CSS selectors and XPath (it's quite common to see XPath "cheat sheets" online that show equivalent CSS selectors; a lot of developers are more used to thinking in CSS rather than XPath).

The node test portion of localization we've just looked at allows for a basic level of filtering (by element name, by type), but for precision filtering, we need to add a predicate to our localization step. Predicates are sub-expressions contained in [ ] bracket characters and can be formulated with path expressions, functions, operations, strings, and numbers. XPath predicates can even be nested, used as list indexes, chained, and used as arguments to functions called in other predicates! These features allow for some mightily powerful effects: all CSS selector expressions have XPath equivalents, but the inverse is not true. When a localization step contains a predicate, the nodes that have been selected by the current axis and node test are then tested against the predicate expression, which returns true or false for each node; those that evaluate to true are included in the final list of results.

Let's see how we can use a predicate to filter our XML by attribute:

```
// ... $xml, $dom, and $xpath init as before
$results = $xpath->query(
    '/descendant::project[attribute::name="php"]'
);
print "Found {$results->length} matching node(s)" . PHP_EOL;
var_dump(get_class($results[0]));
var_Dump($results[0]->tagName);
var_Dump($results[0]->textContent);
```

```
Output:
Found 1 matching node(s)
string(11) "Dom\Element"
string(7) "project"
string(10) "Lovely PHP"
```

Perfect! We asked XPath to fetch all `project` nodes that are descendants of the document node and to filter them based on the `name` attribute, specifying that `attribute::name` had to match a value of the string `"php"`. As we can see from the output, we get a single node in the list of results – a `Dom\Element` instance – and the `tagName` and `textContent` confirm it's the correct element from our original XML document.